User Onboarding Validation Module (Python + Pytest)

Implementation of custom exceptions, registration validation service, regex-based email validation, age restriction enforcement, internal assertions, and a pytest test suite using fixtures and pytest.raises.

registration_service.py

```
import re

class InvalidEmailError(ValueError):
    def __init__(self, email):
        super().__init__(f"Invalid email address: '{email}'")


class UnderageError(PermissionError):
    def __init__(self, age):
        super().__init__(f"User must be at least 18 years old. Provided age: {age}")


class RegistrationService:
    EMAIL_PATTERN = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"

    def register_user(self, email: str, age: int) -> bool:
        # Internal invariant check
        assert isinstance(age, int), "Age must be an integer"

        if email is None or email.strip() == "":
            raise InvalidEmailError(email)

        if not re.match(self.EMAIL_PATTERN, email):
            raise InvalidEmailError(email)

        if age < 18:
            raise UnderageError(age)

        return True
```

test_registration_service.py

```
import pytest
from registration_service import (
    RegistrationService,
    InvalidEmailError,
    UnderageError
)

@pytest.fixture
def service():
    return RegistrationService()


def test_successful_registration(service):
    assert service.register_user(
        "john.doe@example.com",
        25
    ) is True


def test_invalid_email_empty(service):
    with pytest.raises(InvalidEmailError):
        service.register_user("", 25)


def test_invalid_email_none(service):
    with pytest.raises(InvalidEmailError):
        service.register_user(None, 25)


def test_invalid_email_format(service):
    with pytest.raises(InvalidEmailError):
        service.register_user("invalid-email", 25)


def test_underage_user(service):
    with pytest.raises(UnderageError):
        service.register_user(
            "student@example.com",
            17
        )


def test_assertion_for_invalid_age_type(service):
    with pytest.raises(AssertionError):
        service.register_user(
            "user@example.com",
            "twenty"
        )
```